

Exercise 1c: Inverse Kinematics of the ABB IRB 120

Prof. Marco Hutter*

Teaching Assistants:
Arjun Bhardwaj, Kissan Varatharajan, Thomas Li

October 7, 2026

Abstract

The aim of this exercise is to calculate the inverse kinematics of an ABB robot arm. To do this, you will have to implement a pseudo-inversion scheme for generic matrices. You will also implement a simple motion controller based on the kinematics of the system. A separate MATLAB script will be provided for the 3D visualization of the robot arm.



Figure 1: The ABB IRW 120 robot arm.

*original contributors include Michael Blösch, Dario Bellicoso, and Samuel Bachmann

1 Introduction

The following exercise is based on an ABB IRB 120 depicted in Fig. 1. It is a 6-link robotic manipulator with a fixed base. During the exercise you will implement several different MATLAB functions, which, you should test carefully since the following tasks are often dependent on them. To help you with this, we have provided the script prototypes together with a visualizer of the manipulator.

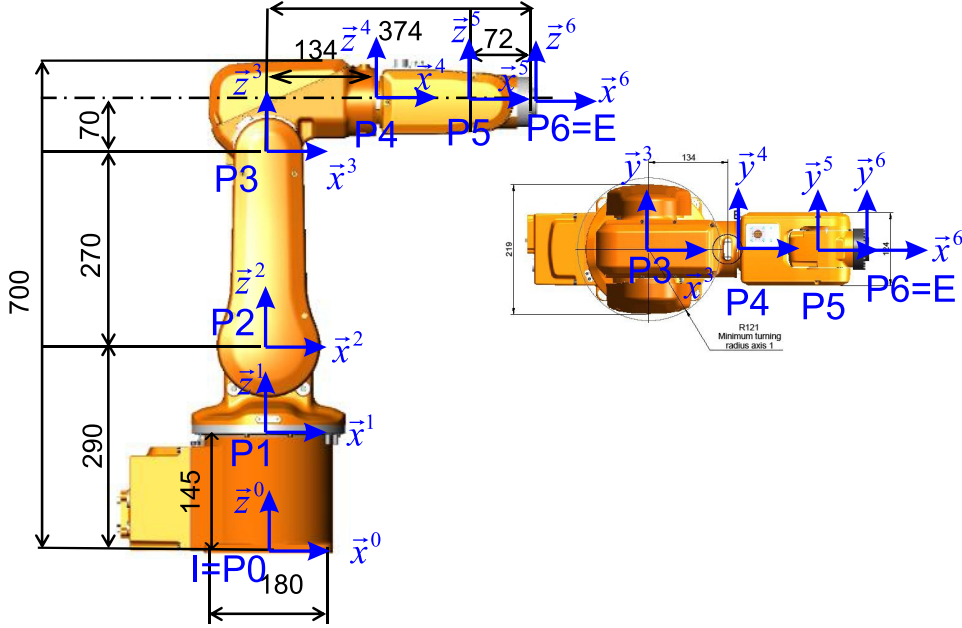


Figure 2: ABB IRB 120 with coordinate systems and joints

Throughout this document, we will employ I for denoting the inertial world coordinate system (which has the same pose as the coordinate system P0 in figure 2) and E for the coordinate system attached to the end-effector (which has the same pose as the coordinate system P6 in Fig. 2).

2 Matrix Pseudo-Inversion

The *Moore-Penrose pseudo-inverse* is a generalization of the *matrix inversion* operation for non-square matrices. Let a non-square matrix A be defined in $\mathbb{R}^{m \times n}$. When $m \geq n$ and $\text{rank}(A) = n$, it is possible to define the so-called *left pseudo-inverse* A_l^+ as

$$A_l^+ := (A^T A)^{-1} A^T, \quad (1)$$

which yields $A_l^+ A = \mathbb{I}_{n \times n}$. If instead it is $m \leq n$ and $\text{rank}(A) = m$, then it is possible to define the *right pseudo-inverse* A_r^+ as

$$A_r^+ := A^T (A A^T)^{-1}, \quad (2)$$

which yields $A A_r^+ = \mathbb{I}_{m \times m}$. If one wants to handle singularities, then it is possible to define a *damped pseudo-inverse* with damping factor λ as

$$A_l^+ := (A^T A + \lambda^2 \mathbf{I}_{n \times n})^{-1} A^T, \quad (3)$$

and

$$A_r^+ := A^T (A A^T + \lambda^2 \mathbf{I}_{m \times m})^{-1}. \quad (4)$$

Note that for square and invertible matrices, the pseudo-inverse is equivalent to the usual matrix inverse.

Exercise 2.1

In this first exercise, you are required to provide an implementation of (3) and (4) as a MATLAB function. The function place-holder to be completed is:

Listing 1: `pseudoInverseMat.m`

```
1 function [ pinvA ] = pseudoInverseMat(A, lambda)
2 % Input: Any m-by-n matrix.
3 % Output: An n-by-m pseudo-inverse of the input according to the ...
4           Moore-Penrose formula
5 % Get the number of rows (m) and columns (n) of A
6 [m, n] = size(A);
7
8 % TODO: complete the computation of the pseudo-inverse.
9 % Hint: How should we account for both left and right ...
10        pseudo-inverse forms?
11 pinvA = zeros(n, m);
12 end
```

3 Iterative Inverse Kinematics

Consider a desired position $\mathbf{x}_{IE}^* = [0.5649 \ 0 \ 0.5509]^T$ and orientation $\mathbf{C}_{IE}^* = \mathbf{I}_{3 \times 3}$ which shall be jointly called pose χ_e^* . We wish to find the joint space configuration $\mathbf{q}$ which corresponds to the desired pose. This exercise focuses on the implementation of an iterative inverse kinematics algorithm, which can be summarized as follows:

1. $\mathbf{q} \leftarrow \mathbf{q}^0$ ▷ start configuration
2. while $\|\chi_e^* \boxminus \chi_e(\mathbf{q})\| > tol$ ▷ while the solution is not reached
3. $\mathbf{J}_{e0} \leftarrow \mathbf{J}_{e0}(\mathbf{q})$ ▷ evaluate Jacobian for current $\mathbf{q}$
4. $\mathbf{J}_{e0}^+ \leftarrow (\mathbf{J}_{e0})^+$ ▷ update the pseudoinverse
5. $\Delta\chi_e \leftarrow \chi_e^* \boxminus \chi_e(\mathbf{q})$ ▷ find the end-effector configuration error vector
6. $\mathbf{q} \leftarrow \mathbf{q} + \alpha \mathbf{J}_{e0}^+ \Delta\chi_e$ ▷ update the generalized coordinates (step size α)

Note that we are using the geometric Jacobian $\mathbf{J}_{e0}$, which was derived in the last exercise. The boxminus ($\boxminus$) operator is a generalized difference operator that allows “subtraction” of poses. The orientation difference is thereby defined as the rotational vector extracted from the relative rotation between the desired orientation $\mathbf{C}_{IE}^*$ and the one based on the solution of the current iteration $\mathbf{C}_{IE}(\mathbf{q})$, i.e.,

$$\Delta\varphi = {}_I\varphi_{EE*} = \text{rotMatToPhi}(\mathbf{C}_{IE}^* \mathbf{C}_{IE}^T(\mathbf{q})). \quad (5)$$

Exercise 3.1

Your task is to implement the iterative inverse kinematics algorithm by completing the following two Matlab functions. Use `rotMatToRotVec` as a helper function to calculate the pose error.

Note: Your implementation should be robust against the case for which the rotation is identity, i.e. the rotation angle is zero.

Note2: You can test the `inverseKinematics` function by calling it with arguments of your choice.

Listing 2: rotMatToRotVec.m

```

1 function [ phi ] = rotMatToRotVec(C)
2 % Input: a rotation matrix C
3 % Output: the rotational vector which describes the rotation C
4
5 % Compute the rotational vector
6 phi = zeros(3,1);
7 end

```

Listing 3: inverseKinematics.m

```

1 function [ q ] = inverseKinematics(I_r,IE_des, C,IE_des, q_0, tol)
2 % Input: desired end-effector position, desired end-effector ...
3 %           orientation (rotation matrix),
4 %           initial guess for joint angles, threshold for the ...
5 %           stopping-criterion
6 % Output: joint angles which match desired end-effector position ...
7 %           and orientation
8
9 % 0. Setup
10 it = 0;
11 max_it = 100;           % Set the maximum number of iterations.
12 lambda = 0.001;        % Damping factor
13 alpha = 0.5;           % Update rate
14
15 close all;
16 loadviz;
17
18 % 1. start configuration
19 q = q_0;
20
21 % 2. Iterate until terminating condition.
22 while (it==0 || (norm(dxe)>tol && it < max_it))
23     % 3. evaluate Jacobian for current q
24     I_J = ;
25
26     % 4. Update the psuedo inverse
27     I_J.pinv = ;
28
29     % 5. Find the end-effector configuration error vector
30     % position error
31     dr = ;
32     % rotation error
33     dph = ;
34     % pose error
35     dxe = ;
36
37     % 6. Update the generalized coordinates
38     q = ;
39
40     % Update robot
41     abbRobot.setJointPositions(q);
42     drawnow;
43     pause(0.1);
44
45     it = it+1;
46 end
47
48 % Get final error (as for 5.)
49 % position error
50 dr = ;
51 % rotation error
52 dph = ;
53
54 fprintf('Inverse kinematics terminated after %d iterations.\n', it);
55 fprintf('Position error: %e.\n', norm(dr));

```

```

53 fprintf('Attitude error: %e.\n', norm(dph));
54 end

```

4 Kinematic Motion Control

The final section in this problem set will demonstrate the use of the iterative inverse kinematics method to implement a basic end-effector pose controller for the ABB manipulator. The controller will act only on a kinematic level, i.e. it will produce end-effector velocities as a function of the current and desired end-effector pose. This will result in a motion control scheme which should track a series of points defining a trajectory in the task-space of the robot. For all of this to work we will additionally need the following functional modules:

1. A trajectory generator, which will produce an 3-by-N array, containing N points in Cartesian space defining a discretized path that the end-effector should track.
2. A kinematics-level simulator, which will integrate over each time-step, the resulting velocities generated by the kinematic motion controller of the previous exercise. This integration, at each iteration, should generate an updated configuration of the robot which is then provided to the visualization for rendering.

To save time during the exercise session, we have provided functions to implement most of the grunt work regarding the aforementioned points. Execute and inspect the `motion_control_visualization.m` function. It will start the motion control simulation using inputs from your motion controller (which will be implemented in `kinematicMotionControl.m`). The animation and corresponding plots will visualize the performance of your controller. The function `generateLineTrajectory.m` generates a straight-line trajectory defined between two points for a given path duration and time step size.

Listing 4: `motion_control_visualization.m`

```

1 function [] = motion_control_visualization()
2 % Motor control visualization script
3
4 % ===== Trajectory settings =====
5 ts = 0.05; % Set the sampling time (in ...
    seconds)
6 r_start = [0.4 0.1 0.6].'; % 3x1 (m)
7 r_end = [-0.4 0.3 0.5].'; % 3x1 (m)
8 v_line = 0.4; % 1x1 (m/s)
9 q_0 = zeros(6,1); % 6x1 (rad)
10 use_solution = 1; % 0: user implementation, 1: ...
    solution
11 % =====
12
13 % Load the visualization
14 f1 = figure(1); close(f1); loadviz;
15
16 % Initialize the vector of generalized coordinates
17 q = q_0;
18 abbRobot.setJointPositions(q);
19
20 % Generate a new desired trajectory
21 dr = r_end - r_start;
22 tf = norm(dr)/v_line; % Total trajectory time
23 N = floor(tf/ts); % Number time steps

```

```

24 t = ts*(1:N);
25 r_traj = generateLineTrajectory(r.start, r.end, N);
26 v_traj = repmat(v_line * (dr) ./ norm(dr), N, 1); % Constant ...
    velocity reference
27 r_log = NaN*zeros(size(r_traj));
28 v_log = NaN*zeros(size(v_traj));
29
30 % Plot real trajectory
31 figure(2); clf; hold all
32 r_h = plot(t, r_log);
33 for i = 1:3
34     plot(t, r_traj(:,i), '--', 'Color', get(r_h(i), 'Color'));
35 end
36 title('End effector position in Inertial frame')
37 legend({'x', 'y', 'z', 'x-{ref}', 'y-{ref}', 'z-{ref}'})
38
39 figure(3); clf; hold all
40 v_h = plot(t, v_log);
41 for i = 1:3
42     plot(t, v_traj(:,i), '--', 'Color', get(r_h(i), 'Color'));
43 end
44 title('End effector linear velocity in Inertial frame')
45 legend({'x', 'y', 'z', 'x-{ref}', 'y-{ref}', 'z-{ref}'})
46
47 % Notify that the visualization loop is starting
48 disp('Starting visualization loop. ');
49 pause(0.5);
50
51 % Run a visualization loop
52 for k = 1:N
53     startLoop = tic; % start time counter
54
55     % Get the velocity command
56     switch use_solution
57         case 0
58             Dq = kinematicMotionControl(q, r_traj(k,:), '...', ...
                v_traj(k,:));
59         case 1
60             Dq = kinematicMotionControl_solution(q, r_traj(k,:), '...', ...
                v_traj(k,:));
61     end
62
63     % Time integration step to update visualization. This would ...
        also be used for a position controllable robot
64     q = q + Dq*ts;
65
66     % Set the generalized coordinates to the robot visualizer class
67     abbRobot.setJointPositions(q);
68     r_log(k,:) = jointToPosition_solution(q);
69     v_log(k,:) = jointToPosJac_solution(q)*Dq;
70
71     % Update the visualizations
72     for i = 1:3
73         set(r_h(i), 'Ydata', r_log(:,i));
74         set(v_h(i), 'Ydata', v_log(:,i));
75     end
76     drawnow;
77
78     % If enough time is left, wait to try to keep the update frequency
79     % stable
80     residualWaitTime = ts - toc(startLoop);
81     if (residualWaitTime > 0)
82         pause(residualWaitTime);
83     end
84 end
85
86 % Notify the user that the script has ended.

```

```

87 disp('Visualization loop has ended.');
```

```

88
89
90 end
91
92 function [ r_traj ] = generateLineTrajectory(r_start, r_end, N)
93 % Inputs:
94 %     r_start : start position
95 %     r_end   : end position
96 %     N       : number of timesteps
97 % Output: Nx3 matrix End-effector position reference
98 x_traj = linspace(r_start(1), r_end(1), N);
99 y_traj = linspace(r_start(2), r_end(2), N);
100 z_traj = linspace(r_start(3), r_end(3), N);
101 r_traj = [x_traj; y_traj; z_traj].';
102 end

```

Exercise 4.1

The final exercise that combines the tools in the previous questions is to implement a kinematic controller that tracks a single 3D line trajectory. When implementing `kinematicMotionControl.m`, you can play around with the trajectory settings at the top of the file. Investigate what happens if you specify a pose that the robot cannot reach.

Listing 5: `kinematicMotionControl.m`

```

1 function [ Dq ] = kinematicMotionControl(q, r_des, v_des)
2 % Inputs:
3 % q       : current configuration of the robot
4 % r_des   : desired end effector position
5 % v_des   : desired end effector velocity
6 % Output: joint-space velocity command of the robot.
7
8 % Compute the updated joint velocities. This would be used for a ...
    velocity controllable robot
9 % TODO:
10 Dq = 0.1*ones(6,1);
11 end

```